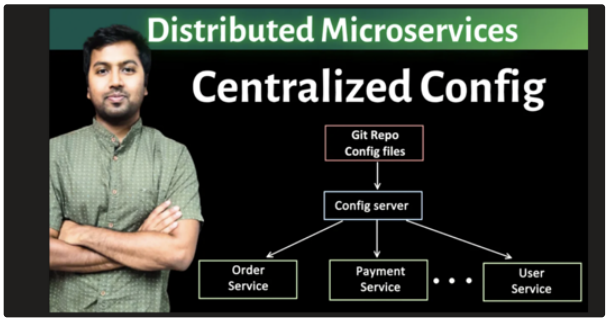
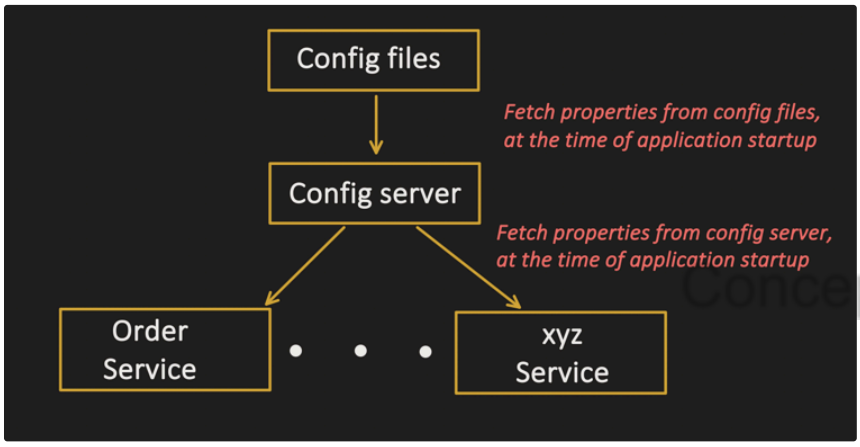
@RefreshScope

We have already covered Centralized Configuration:





Concept && Coding

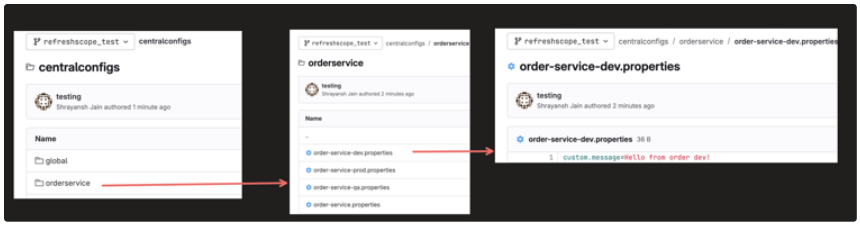
And we finished with asking 1 question that:

Say, our Order application is running and someone has updated the Central config for order service, do I need to restart the app every time?

Answer is NO, and its solution is **@RefreshScope**

First, lets try to re-produce this scenario:

<u>Git Repo for CentralConfigs:</u>



ConfigServer:

pom.xml

```
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-config-server</artifactId>
</dependency>

```

```
@SpringBootApplication
@EnableConfigServer
public class ConfigServerApplication {

    public static void main(String[] args) {
        SpringApplication.run(ConfigServerApplication.class, args);
    }
}

```

Concept && Coding

```
server.port=8888

spring.cloud.config.server.git.uri=https://gitlab.com/shrayansh8/centralconfigs
spring.cloud.config.server.git.username=${GIT_USERNAME}
spring.cloud.config.server.git.password=${GIT_ACCESS_TOKEN}
spring.cloud.config.server.git.search-paths=global, orderservice
spring.cloud.config.server.git.clone-on-start=true
spring.cloud.config.server.git.default-label=refreshscope_test
```

Order Microservice:

pom.xml

```
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-config</artifactId>
</dependency>

```

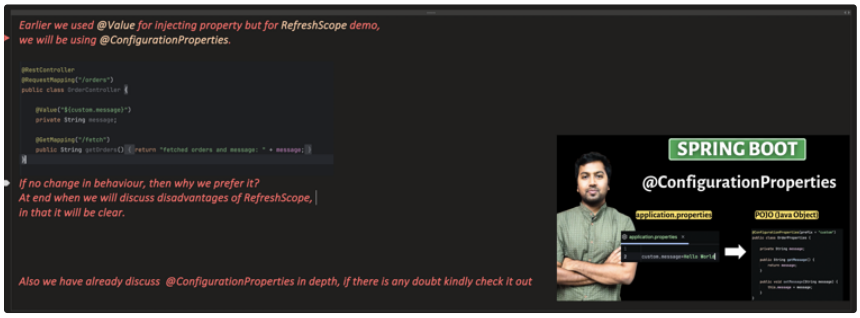
application.properties

```
spring.application.name=order-service
spring.config.import=optional:configserver:http://localhost:8888
spring.profiles.active=dev

#config for all profile (dev, qa, prod etc)
custom.message=Hello from local default!

```

```java
@Component
@ConfigurationProperties(prefix = "custom")
public class OrderProperties {

    private String message;

    public String getMessage() {
        return message;
    }

    public void setMessage(String message) {
        this.message = message;
    }
}

```

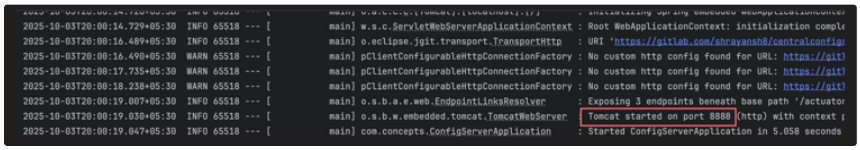
```java
@RestController
@RequestMapping("/orders")
public class OrderController {

    private int counter = 0;

    @Autowired
    OrderProperties orderProperties;

    @GetMapping("/fetch")
    public String getOrders() {
        counter++;
        return "fetched orders and message: " + orderProperties.getMessage()
                + " and counter value is: " + counter;
    }
}

```

Concept && Coding

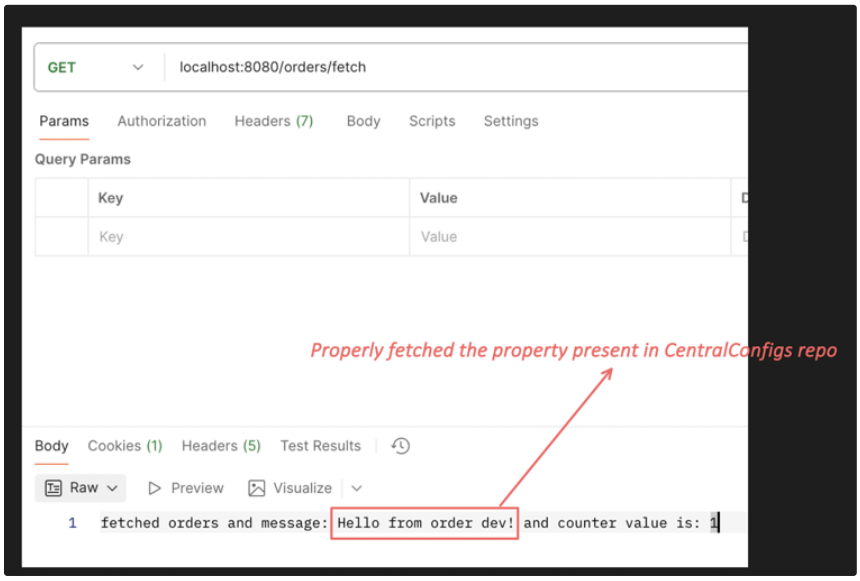


Let's start both ConfigServer and Order Server:

1. **ConfigServer :**

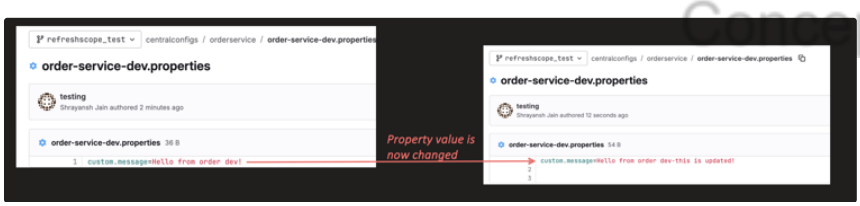


2. **Order Server:**



Now lets update the CentralConfigs property :

Concept && Coding



Now, lets hit the Order API again:

@RefreshScope, help to resolve this:

- It helps to mark the bean, which need to be destroyed and re-created (with latest value) when refresh event occurs. like below:

```
@Component
@ConfigurationProperties(prefix = "custom")
@RefreshScope
public class OrderProperties {

    private String message;

    public String getMessage() {
        return message;
    }

    public void setMessage(String message) {
        this.message = message;
    }
}

```

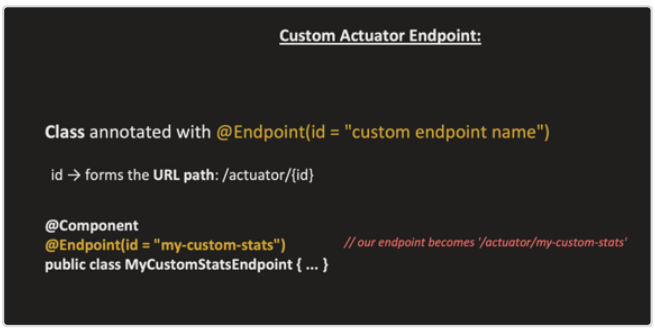
But now question is: **How refresh event will occur, so that my beans (which are marked with @RefreshScope) its fields get updated with latest property value?**

One way is: **Actuator**

We have already covered this topic in-depth in Springboot series, kindly check it out, if you have any doubt:

Concept && Coding

If we can recall, we covered Custom Actuator Endpoint within Actuator topic, like how to create our own Actuator Endpoint.



Spring Cloud Config make uses of this functionality and provided us one custom actuator endpoint: "***/actuator/refresh***"

```java
/*
    Id: forms the URL path: /actuator/{id}
*/
@Endpoint(id = "refresh")
public class RefreshEndpoint {

    private static final Log LOG = LogFactory.getLog(RefreshEndpoint.class);

    private final ContextRefresher contextRefresher;

    public RefreshEndpoint(ContextRefresher contextRefresher) {
        this.contextRefresher = contextRefresher;
    }

    @WriteOperation
    public Collection<String> refresh() {
      /*
    @WriteOperation is similar to POST Operation
        1. If its Config Server: it fetches the properties from CentralConfigs
        Repo

      2. If its Config Client like Order Service: it will invoke the ConfigServer
    , fetches the properties again and then look for @RefreshScope marked beans
      and re-create them with fresh property values.
        */
        Set<String> keys = this.contextRefresher.refresh();
        LOG.info("Refreshed keys : " + keys);
        return keys;
    }
}

```

Concept && Coding

All we need to do is add the Actuator dependency:

ConfigServer:

pom.xml

```xml
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-config-server</artifactId>
</dependency>

```

```xml
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>

```

application.properties

```
server.port=8888

```

```
spring.cloud.config.server.git.uri=https://gitlab.com/shrayansh8/centralconfigs
spring.cloud.config.server.git.username=${GIT_USERNAME}
spring.cloud.config.server.git.password=${GIT_ACCESS_TOKEN}
spring.cloud.config.server.git.search-paths=global, orderservice
spring.cloud.config.server.git.clone-on-start=true
spring.cloud.config.server.git.default-label=refreshscope_test

management.endpoints.web.exposure.include=refresh,health

```

Order Microservice:

pom.xml

```
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-config</artifactId>
</dependency>

```

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>

```

application.properties

```
spring.application.name=order-service
spring.config.import=optional:configserver:http://localhost:8888
spring.profiles.active=dev

#config for all profile (dev, qa, prod etc)
custom.message=Hello from local default!

management.endpoints.web.exposure.include=health,refresh

```

Concept && Coding

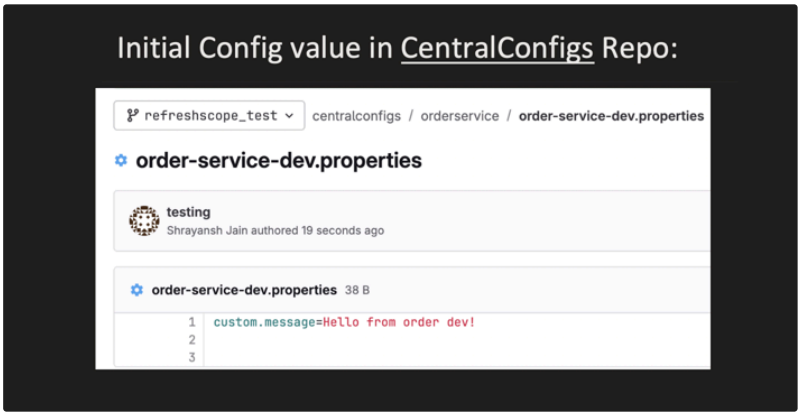
```
@Component
@ConfigurationProperties(prefix = "custom")
@RefreshScope
public class OrderProperties {

    private String message;

    public String getMessage() {
        return message;
    }

    public void setMessage(String message) {
        this.message = message;
    }
}

```

(No change in controller)

```
@RestController
@RequestMapping("/orders")
public class OrderController {

    private int counter = 0;
```

```
6
7     @Autowired
8     OrderProperties orderProperties;
9
10    @GetMapping("/fetch")
11    public String getOrders() {
12        counter++;
13        return "fetched orders and message: " +
orderProperties.getMessage() +
14                " and counter value is: " + counter;
15    }
16 }
17
```

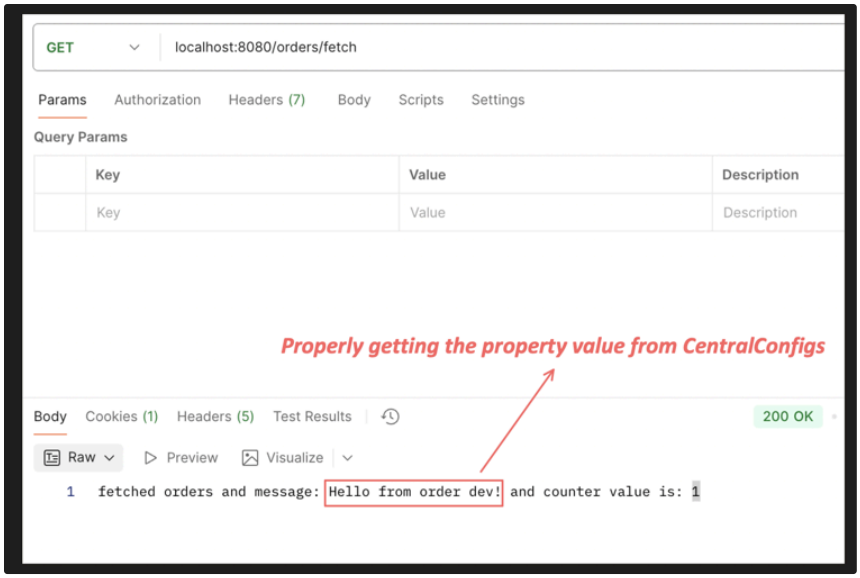
Let's start the servers again and test now:



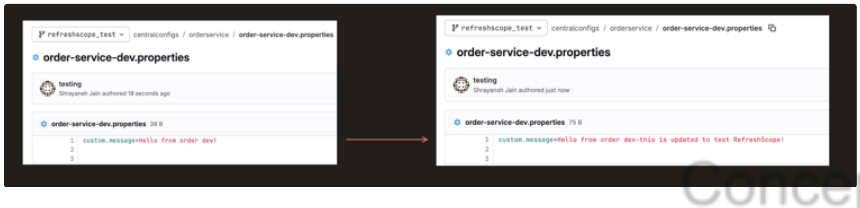
Concept && Coding

ConfigServer:

Order Microservice:

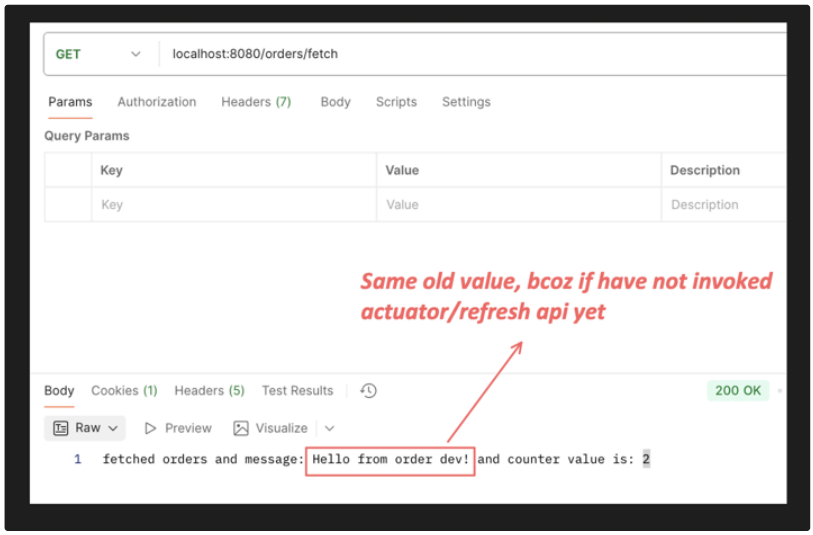


Now Update the property in CentralConfigs :

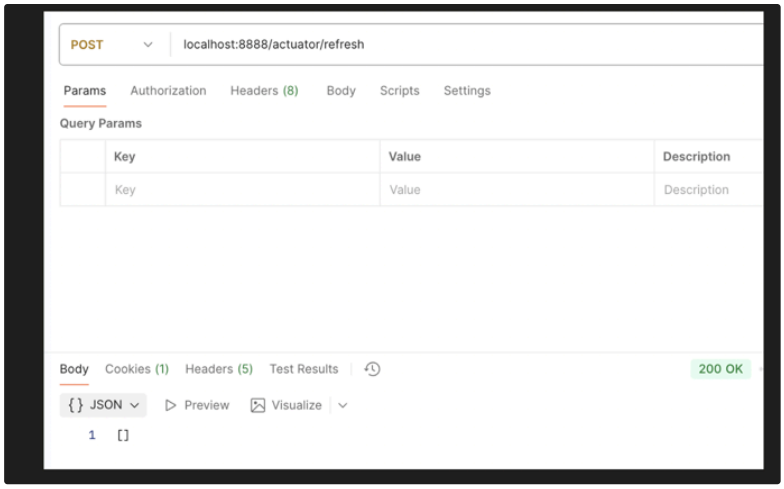


Concept && Coding

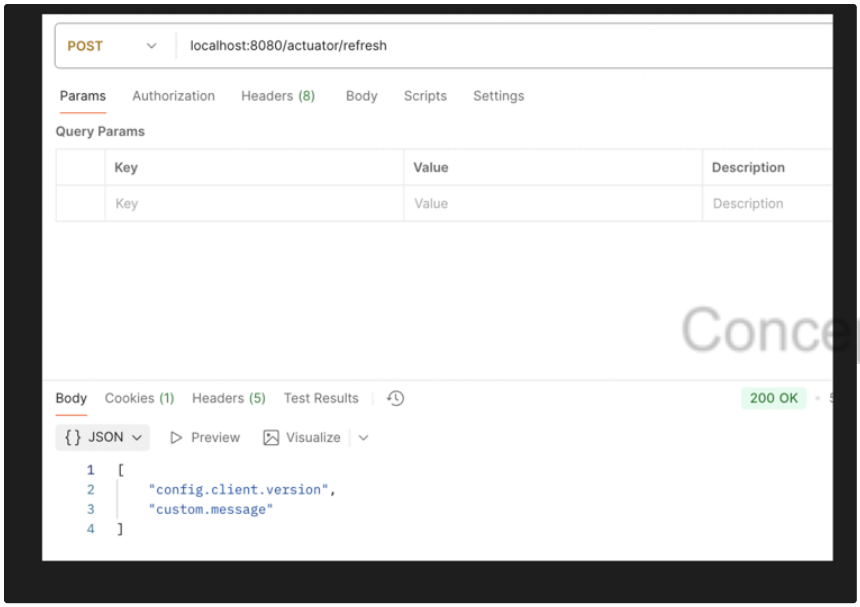
Now, first try to hit the Order API again:



Now, first invoke **"/actuator/refresh"** endpoint on ConfigServer

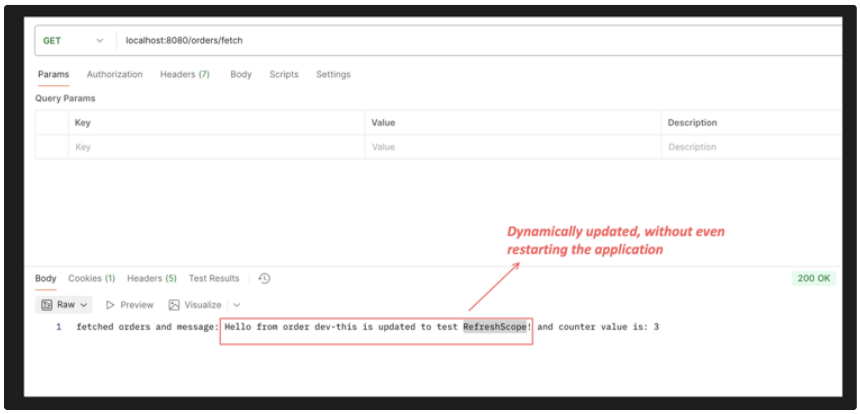


Now, invoke "**/actuator/refresh**" endpoint on Order Service:



Concept && Coding

Now, try to hit the "/fetch" Order API again:



Now lets comeback to point: Why we used @ConfigurationProperties?

- Remember, when bean is refreshed its first destroyed and then recreated.
- So you will loose your object state.

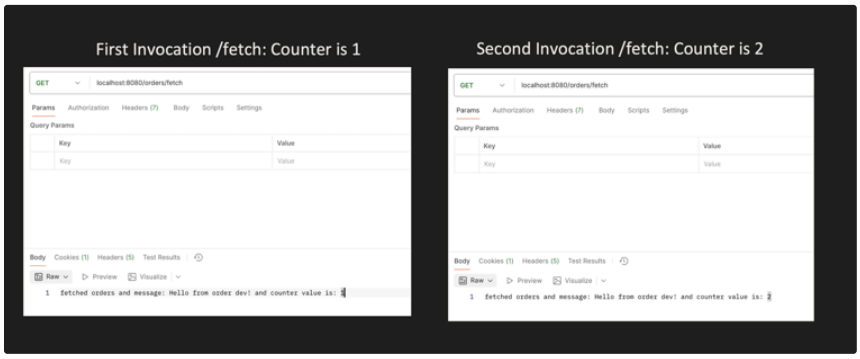
That's, why its never recommended to use @RefreshScope over Controller or Service. Its not safe.

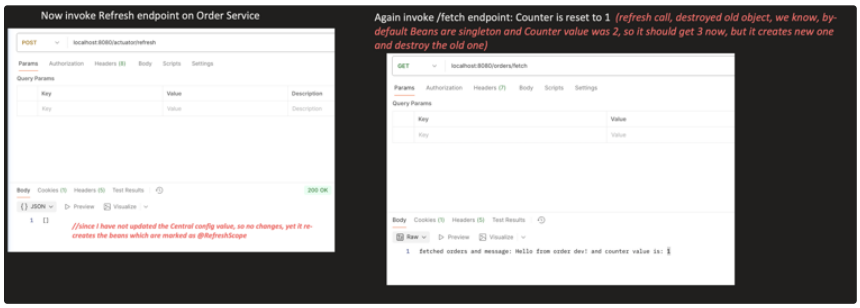
@ConfigurationProperties are stateless, just properties class, so its safe to use it above it.

Understand with below example:

```
@RestController
@RequestMapping("/orders")
@RefreshScope
public class OrderController {

    private int counter = 0;

    @Value("${custom.message}")
    String message;

    @GetMapping("/fetch")
    public String getOrders() {
        counter++;
        return "fetched orders and message: " + message +
                " and counter value is: " + counter;
    }
}

```

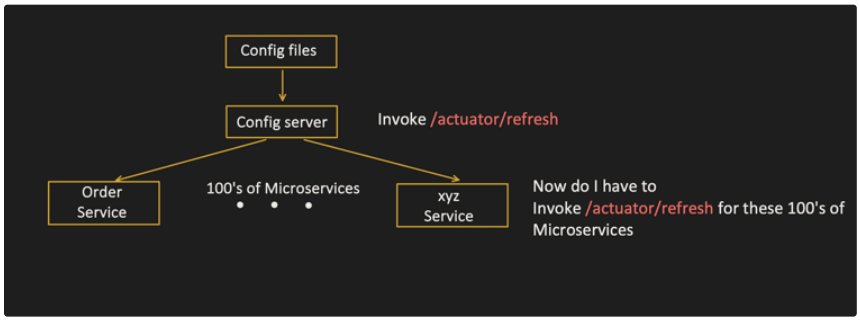
Concept && Coding





That's the reason for @RefreshScope, I used @ConfigurationProperties and used @RefreshScope there instead of using at Controller class.

Okay, now 1 last question might have come to your mind:



If we are using **Actuator** approach to create a **refresh** event then YES.

But, if we want , only if I do **REFRESH** at **ConfigServer** and all the microservices get notified and they should also refresh themselves.

Then its also possible with "**Spring Cloud Bus**", will see later.

Concept && Coding